A Framework for Joint Wireless and Plug-In Charger Placement with Reinforcement Learning in MATSim

1st Isaac Peterson
Electrical and Computer Engineering
Utah State University
Logan, Utah
isaac.peterson@usu.edu

2nd Mario Harper
Electrical and Computer Engineering
Utah State University
Logan, Utah
mario.harper@usu.edu

***Abstract*—To accelerate the adoption of electric vehicles (EVs), a well-developed and strategically placed charging infrastructure is essential. Traditionally, plug-in charging has been the dominant model. However, emerging technologies in wireless charging, which enable EVs to charge while driving, are reshaping the possibilities for infrastructure planning. Although substantial research has focused on optimizing the placement of either plug-in or wireless chargers, few approaches address the placement of both types concurrently. This work fills that gap by formulating and solving a comprehensive Optimal Charger Placement (OCP) problem that considers the trade-offs between the two charger types. We extract a real-world road network and simulate EV traffic using MATSim, enabling comparison against empirical traffic flow data. To optimize charger placement, we develop a reinforcement learning framework based on Proximal Policy Optimization (PPO), which learns to balance installation costs, charger efficiency and time saved for EV drivers.**

***Index Terms*—reinforcement learning, electric vehicles, charging, simulation**

I. Introduction

As noted in [1], two main barriers to the adoption of EVs over gasoline vehicles are the lack of charging stations and the limited driving range of EVs. Currently, charging options are mostly limited to plug-in stations, either at home or at public locations. Wireless charging [2] allows EVs to charge while stationary on the road or while in motion. Implementing this technology would improve the charging infrastructure, increase the practical range of EVs, and reduce resistance to their adoption.

The natural next step is understanding where to place these chargers, which leads to the development of the optimal charger placement (OCP) problem. OCP for plug-in charger placement has been formulated in variety of ways. [3] developed a maximal coverage location model that utilized spatial statistics and points of interest to place chargers. Similiarly, [4] developed a methods for predicting public demand for charging stations using multi-agent transport simulator (MATSim) [5], and using these predictions, developed a capacitated maximal coverage location problem (CMCLP) model, that aims to maximize the coverage of the charging demands. [6] developed a method using particle swarm optimization (PSO) that focusing on optimizing for charger placement and size of the charging stations on a single highway. [7] used a two-step process, evaluating the macro and micro level charger placement problem with a hexagon-based greedy approach. Lastly, [8] created a model for predicting the economic and environment cost of placing chargers, along with a genetic algorithm for placing chargers to meet charging demand. A further review of the problem and model approaches is covered in [9].

In recent years, wireless charging has emerged as a promising alternative to traditional plug-in charging stations. This technology enables EVs to recharge while in motion, introducing new considerations that differentiate it from the conventional plug-in charger placement problem. Consequently, it has spurred a distinct line of research. Several approaches have employed linear and nonlinear programming techniques [10]–[13], with objectives ranging from selecting an optimal subset of candidate locations to maximize captured traffic flow on a transportation network [10], to domain-specific applications such as optimizing charger placement for public transit systems [13]. Additionally, [14] utilizes particle swarm optimization (PSO) to determine wireless charger locations, aiming to minimize infrastructure cost while ensuring adequate vehicle charge levels, a reward structure closely aligned with the one proposed in our method. A comprehensive survey of current methodologies and future directions in this field is provided in [2].

To the best of the authors knowledge, this is the first approach that jointly solves the problem of wireless and plug-in charger placement simultaneously. Along with that, limited to no research has been published on the application of reinforcement learning to solve the optimal charging placement problem.

The main contributions of this paper are as follows:

1) A reinforcement learning framework for optimal charger placement, utilizing MATSim for more realistic charging simulations
2) Development on the wireless charging simulation capabilities of MATSim

3) A solution for jointly solving the wireless and plug-in charger placement problem
4) Open source code and data for further development in the field of optimal charger placement

II. Methodology

A. MATSim

MATSim is a traffic flow simulation software for simulating realistic traffic flow patterns. MATSim has already been proven effective in simulating ride-sharing [15], [16], analyzing the traffic congestion effects of autonomous taxis [17], and even in solving the OCP problem by predicting charger demand [4]. At the time of developing this work, the base electric vehicle (EV) contribution that existed in MATSim worked for simulating plug-in charging only, so we developed the software further to include wireless charging. An overview of the base EV functionality in the MATSim contribution can be found at https://github.com/matsim-org/matsim-libs/tree/main/contribs/ev.

The consumption maps used by the EVs in the simulation are based on work done by [18], and already existed in the EV contribution in MATSim. These consumption maps determine the energy consumption from the vehicle based on the speed of the vehicle and the slope of the link they are traveling on. To incorporate slope into our MATSim network, we used the publically available open elevation API [19]. It should be noted that the open elevation API uses as its reference the ground floor to determine the elevation at any given latitude and longitude, which leads to some discrepancies for certain locations along the road like overpasses etc.

MATSim simulates agents that follow predefined plans across a transportation network. These plans consist of activities that agents reach using a specific mode of transport, which in our case is EV. Agents use shortest path algorithms to determine their routes. Based on the plug-in charging extension, if an agent starts driving on a network link and does not have enough charge to reach the end of the link, the simulation inserts a charging activity into the agent's plan. The agent then reroutes to the nearest plug-in charger. We extended this model to support wireless charging, allowing agents to gain charge while driving on a link equipped with wireless charging. This enables us to evaluate and optimize charger effectiveness.

Stable-baselines 3 [20] supports multi-processing to accelerate the training of reinforcement learning algorithms. In our framework, the main bottleneck is the time required to run MATSim simulations. To fully utilize the multi-processing capabilities and improve scalability, we developed a client-server system, shown in Figure 3. This system enables multiple Java threads to handle requests from separate processes. Each thread runs the corresponding MATSim simulation and returns the resulting rewards, which in our case are charge efficiency and time efficiency.

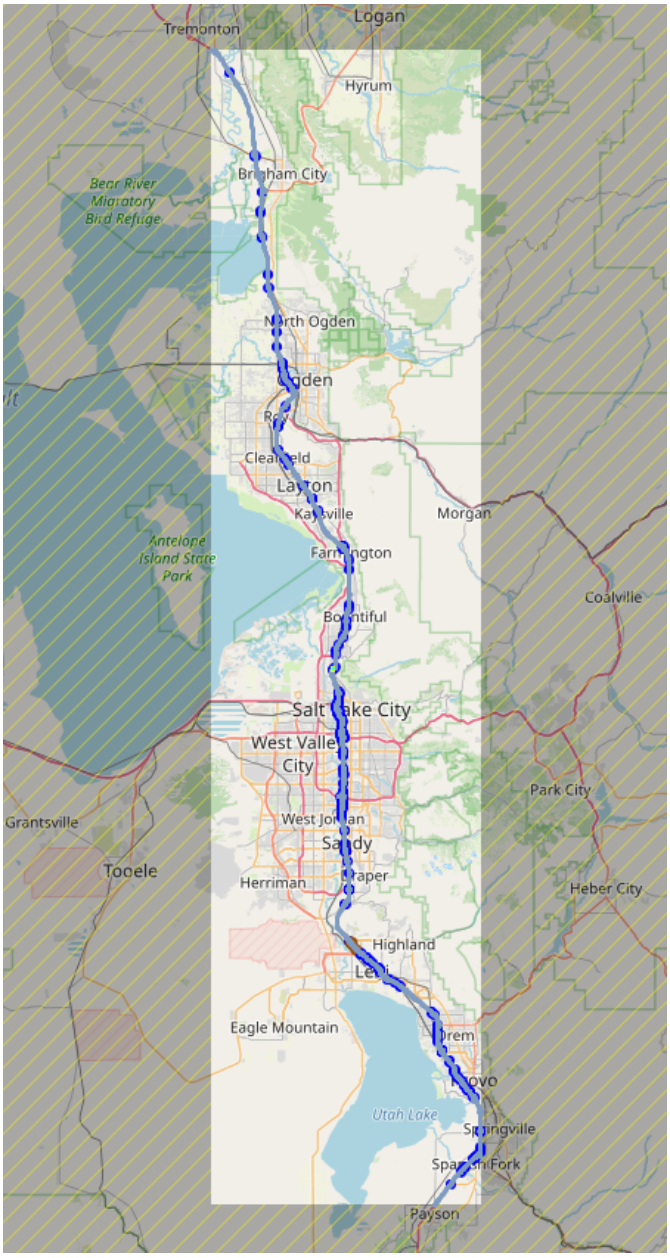

Fig. 1: The network extracted from open street maps (OSM) that we used to solve the OCP problem

B. Traffic flow validation

To ensure the simulated origin-destination paths for our vehicles matched closely with realistic traffic flow data, we generate histograms at each link comparing the simulated MATSim counts with actual traffic flow counts retrieved from the UDOT.

While optimizing the traffic flows to more directly match real data is out of the scope of this work, we utilize a simple gradient based method developed by https://github.com/DIRECTLab/MatsimAI (NOTE: Need a way to cite our previous work here, maybe waiting to see if it's accepted in ITSC). Figure 2 demonstrates the comparison between the real UDOT data and the MATSim simulation

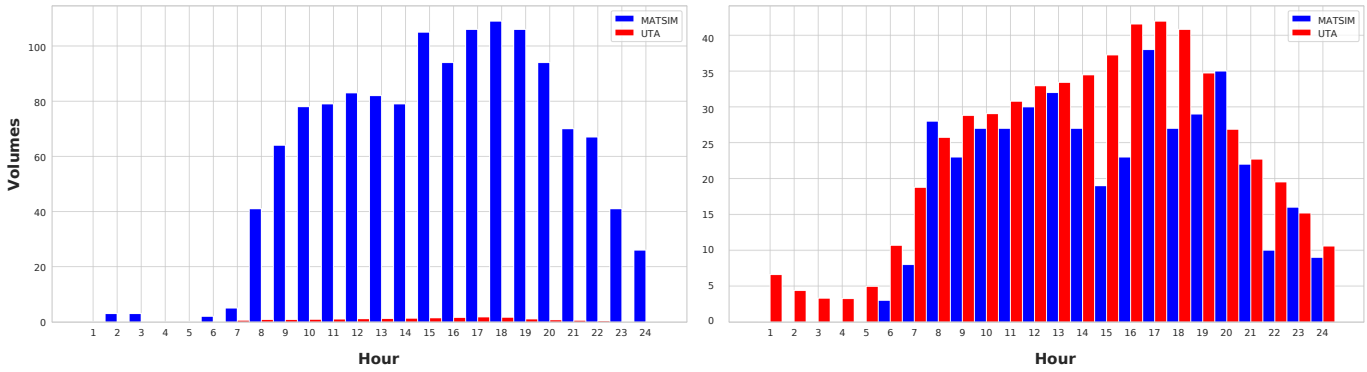

Fig. 2: Based on data obtained by the Utah State government on vehicle registrations in 2025 [21], 155,028 electric vehicles out of a total 9,050,970 vehicles were registered, or around 1.7 % of vehicles. In our experiment, for reasons of scalability and a more representative sample of the number of EV drivers in Utah, we simulated 1 percent of the drivers on I-15. Above shows examples of comparisons of real flow data and simulated results, with the Utah department of transportation (UDOT) data scaled by 0.01, between the worst matched (left), and the best matched (right) links based on mean average percentage error (MAPE). Note that we exclude results where the target value is all zero, as MAPE cannot be applied.

results.

C. Reinforcement Learning

We use the open source python library stable-baselines 3 [20], which uses gymnasium environments [22], simplifying the setup for creating a reinforcement learning model. Extracting the MATSim data into pytorch-geometric graphs [23], we feed these into the proximal policy optimization (PPO) algorithm to optimize for OCP. Each link in our network contains attributes as noted in Table I.

Attribute	Units
Length	meters
Freeway Speed	meters per second
Capacity	# of vehicles
Slope	percent grade
Is None Charger	boolean
Is Wireless Charger	boolean
Is Plug-in Charger	boolean

TABLE I: Link Attributes and their corresponding units.

At each step, the reinforcement learning model receives the full road network as an observation, including all link attributes. The action space has shape $(|E|, 3)$, where $|E|$ is the number of edges in the network. For each edge, the model can choose one of three actions: removing an existing charger if present, place a plug-in charger, or place a wireless charger.

This results in three reward values normalized between 0 and 1, and gives us our final reward function

$$R = C - P - T$$

The reward function optimized by the model consists of three components.

First, to encourage vehicle charge efficiency, we compute the average charge ratio over all time steps:

$$C = \frac{\sum_{t=0}^{T} \hat{c}}{\sum_{t=0}^{T} \max(\hat{c})}$$

where $\hat{c}$ is the average charge of vehicles at time step t, and $\max(\hat{c})$ is the maximum possible charge (battery capacity) for the vehicles.

The second component penalizes the cost of charger installation. We first calculate the maximum possible cost for outfitting the entire road network:

$$p_{\max}(E) = \sum_{e \in E} \max(e_\ell p_d, p_s)$$

where E is the set of links in the network, e_ℓ is the length of link e in kilometers, p_d is the installation cost per kilometer for wireless chargers, and p_s is the fixed cost for plug-in chargers. The actual installation cost for a given network configuration is:

$$p(E) = \sum_{e \in E} \begin{cases} 0 & \text{if } e \text{ has no charger} \\ p_s & \text{if } e \text{ has a plug-in charger} \\ e_\ell p_d & \text{if } e \text{ has a wireless charger} \end{cases}$$

The cost component of the reward is then normalized as:

$$P = \frac{p(E)}{p_{\max}(E)}$$

Following the analysis in [24], which evaluated the costs of different charging systems, we use the upper-bound values for installation costs:

$$p_d = 5,400,000 \text{ USD/km}$$

$$p_s = 136,000 \text{ USD}$$

The third component reflects time efficiency gained from integrating wireless charging. It is calculated as:

$$T = \frac{\sum_{v \in V} d(v)}{|V| \cdot 86400}$$

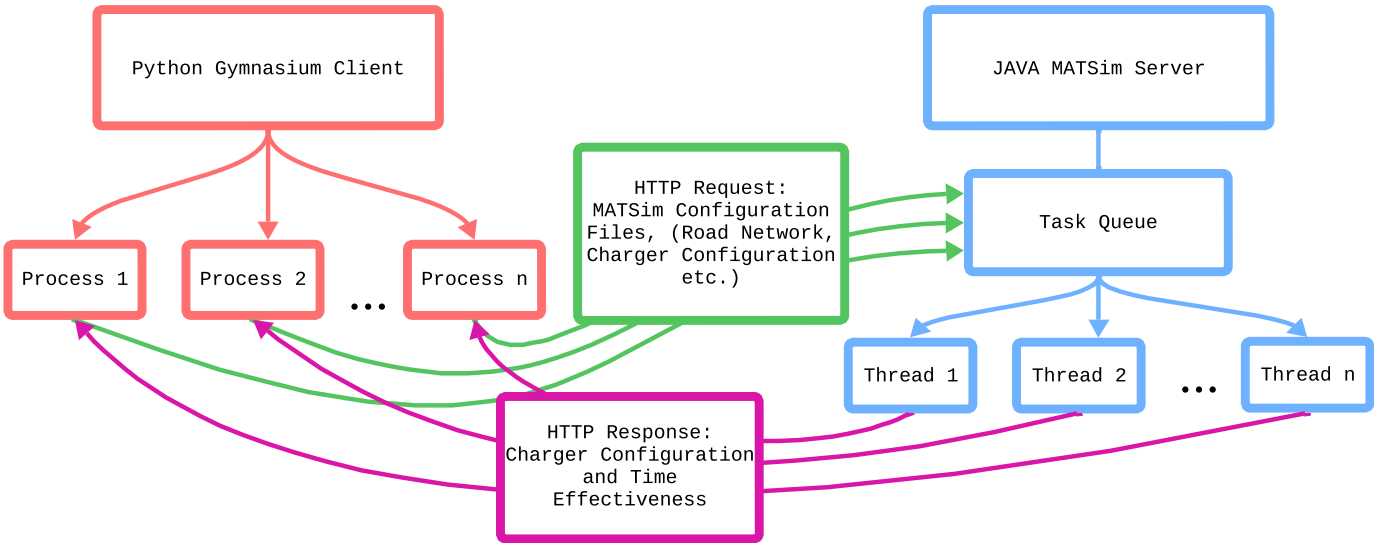

Fig. 3: Overview for the client server system for solving the OCP problem

where V is the set of all vehicles, $d(v)$ is the duration of vehicle v's trip in seconds, and 86400 is the number of seconds in a day. This term captures how quickly trips are completed under a given charger configuration.

Each of the three components, C, P, and T, is normalized between 0 and 1. The final reward is computed as:

$$R = C - P - T$$

III. Results

In this section, we present the results of our proposed reinforcement learning (RL) framework applied to the OCP problem. The primary goal of this work is to develop and evaluate a hybrid method that combines reinforcement learning with high-fidelity traffic simulation software to optimize the placement of EV chargers. Reinforcement learning enables flexible reward function design, allowing the framework to adapt to diverse real-world deployment contexts. The integration of MATSim further enhances the model's realism and configurability, supporting the simulation of heterogeneous vehicles, variable traffic flows, diverse charger types and others [5].

A. Optimization Outcomes Using PPO

The optimization was carried out using the Proximal Policy Optimization (PPO) algorithm. The resulting performance is summarized in Figure 4, which presents a comprehensive overview of the evaluation metrics used to assess the learned policy. Each metric is described in detail in Section II-C.

The experiments were conducted on the I-15 corridor in the state of Utah, a major north-south transportation artery. This corridor was selected due to its high traffic volume and strategic significance in regional mobility. The reinforcement learning agent demonstrated a strong capability to optimize the spatial configuration of EV chargers through successive training episodes.

B. Key Performance Improvements

1) Average Charger Cost: While wireless chargers offer a promising solution to accessibility challenges in EV infrastructure, their installation costs significantly exceed those of conventional plug-in chargers [24]. Accounting for this disparity, the RL model successfully minimized the overall configuration cost by eliminating underutilized chargers and intelligently balancing cost-effectiveness with utility. As shown in Figure 4(a), the final configuration achieves an optimal trade-off between cost and charging coverage.

2) Enhanced Charger Efficiency: A key priority in this study was ensuring that the deployed infrastructure would effectively serve EVs throughout the network. Figure 4(b) demonstrates that the RL agent significantly improved charger efficiency by strategically allocating prospective chargers. This optimization contributes to addressing the persistent bottleneck of underdeveloped charging infrastructure that hampers broader EV adoption [1].

3) Reduction in Waiting Time: The current EV model in MATSim includes a detailed queuing mechanism for plug-in chargers. While plug-in stations are more cost-efficient, they introduce delays due to longer charging times. As seen in Figure 4(c), the RL agent effectively minimized the average waiting time at stations, balancing cost constraints with user experience.

C. Summary of Results

In summary, our hybrid RL-based framework demonstrated robust performance in a realistic traffic scenario. The learned policy effectively adapted to dynamic mobility patterns and infrastructure constraints, resulting in significant improvements across key performance metrics such as charger cost, utilization efficiency, and user wait time. These results affirm the potential of reinforcement learning as a viable approach for tackling complex, multi-objective infrastructure planning

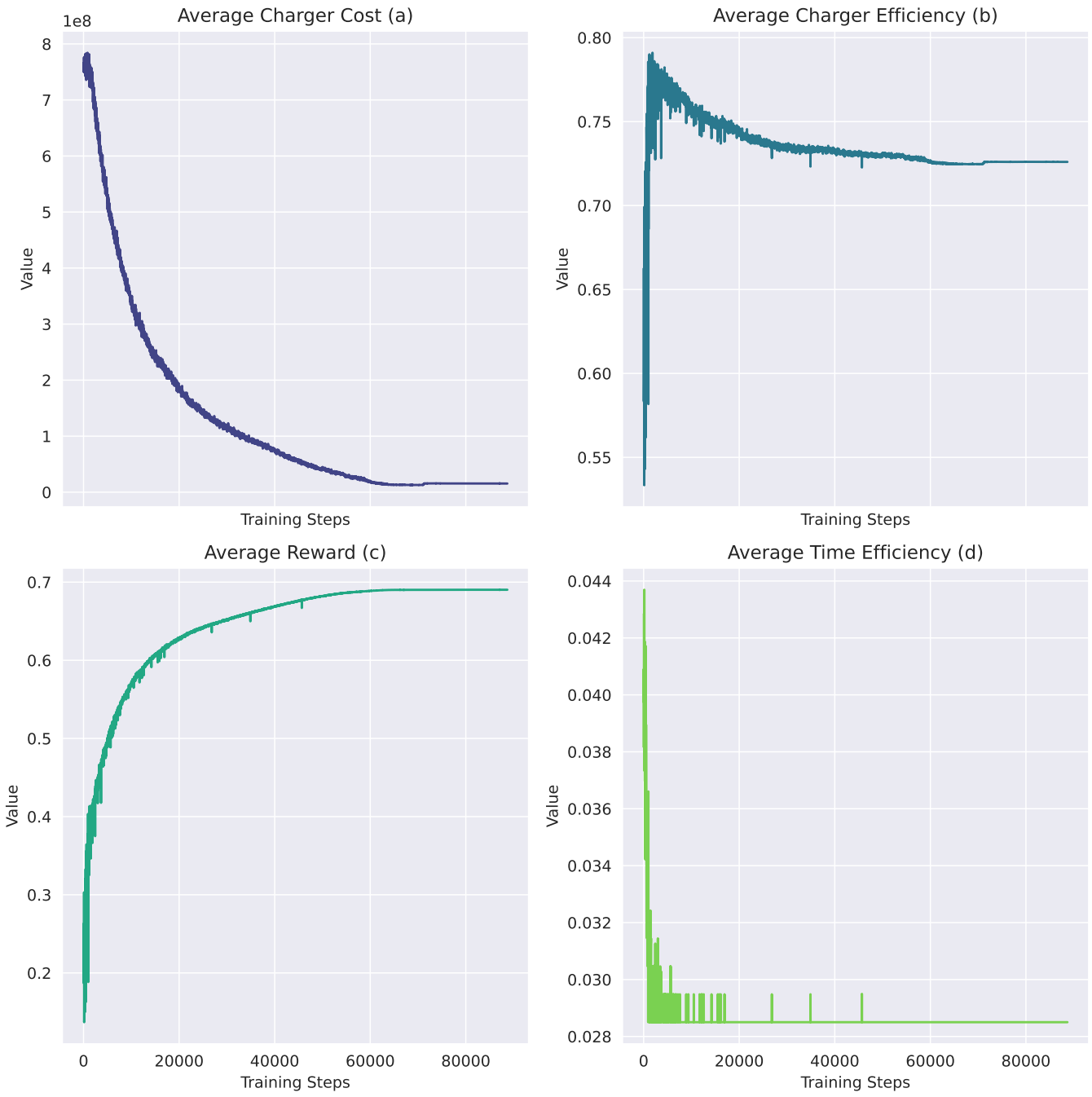

Fig. 4: Results for optimizing OCP with PPO on the I-15 network

problems when integrated with agent-based simulation environments.

IV. Conclusion

In this work, we proposed a hybrid framework that integrates reinforcement learning with realistic traffic simulation to address the OCP (Optimal Charger Placement) problem in electric vehicle (EV) infrastructure planning, considering both wireless and plug-in charging technologies simultaneously. By leveraging the adaptability of reinforcement learning—specifically the PPO (Proximal Policy Optimization) algorithm—we demonstrated that custom reward functions can be effectively utilized to tailor the optimization process to different deployment scenarios and constraints.

Our framework is highly flexible and modular. Although the experiments were conducted on a road network without existing chargers, the architecture allows for the integration of pre-existing infrastructure, enabling more informed and realistic EV infrastructure planning.

Through the integration of MATSim, our approach fa-

cilitated detailed and realistic modeling of traffic networks, vehicle behaviors, and charging demands. We applied the framework to a real-world case study on the I-15 corridor in Utah and showed that the learned policy significantly improved key performance indicators such as waiting time, charger accessibility, and energy satisfaction. These findings confirm that combining reinforcement learning with high-resolution simulation tools provides a powerful and flexible solution for infrastructure optimization challenges.

One major limitation of this work lies in the scalability of the system. Each iteration of the I-15 scenario required several minutes to complete and consumed significant computational memory. Despite using a client-server architecture, the optimization process spanned several days and consumed approximately 100 GB of RAM on average, representing a substantial barrier to accessibility and broader adoption.

Future work could explore the incorporation of multi-agent reinforcement learning, demand forecasting, or dynamic pricing models to further enhance the system's realism and applicability. Moreover, extending the framework to multi-city or regional scenarios could yield valuable insights into long-distance EV travel planning and intercity charger coordination.

REFERENCES

[1] C. Corradi, E. Sica, and P. Morone, "What drives electric vehicle adoption? insights from a systematic review on european transport actors and behaviours," *Energy Research and Social Science*, vol. 95, 1 2023.

[2] Y. J. Jang, "Survey of the operation and system study on wireless charging electric vehicle systems," 10 2018.

[3] G. Dong, J. Ma, R. Wei, and J. Haycox, "Electric vehicle charging point placement optimisation by exploiting spatial statistics and maximal coverage location models," *Transportation Research Part D: Transport and Environment*, vol. 67, pp. 77–88, 2 2019.

[4] Z. Yi, B. Chen, X. C. Liu, R. Wei, J. Chen, and Z. Chen, "An agent-based modeling approach for public charging demand estimation and charging station location optimization at urban scale," *Computers, Environment and Urban Systems*, vol. 101, 4 2023.

[5] A. Horni, K. Nagel, and K. Axhausen, eds., *Multi-Agent Transport Simulation MATSim*. London: Ubiquity Press, Aug 2016.

[6] A. Mohammed, O. Saif, M. A. Abo-Adma, and R. Elazab, "Multiobjective optimization for sizing and placing electric vehicle charging stations considering comprehensive uncertainties," *Energy Informatics*, vol. 7, 12 2024.

[7] C. Csiszár, B. Csonka, D. Földes, E. Wirth, and T. Lovas, "Urban public charging station locating method for electric vehicles based on land use approach," *Journal of Transport Geography*, vol. 74, pp. 173–180, 1 2019.

[8] G. Zhou, Z. Zhu, and S. Luo, "Location optimization of electric vehicle charging stations: Based on cost model and genetic algorithm," *Energy*, vol. 247, 5 2022.

[9] M. O. Metais, J. Suomalainen, Y. Perez, J. Berrada, and E. Suomalainen, "Too much or not enough? planning electric vehicle charging infrastructure: A review of modeling options.," tech. rep., 2021.

[10] R. Riemann, D. Z. Wang, and F. Busch, "Optimal location of wireless charging facilities for electric vehicles: Flow capturing location model with stochastic user equilibrium," *Transportation Research Part C: Emerging Technologies*, vol. 58, pp. 1–12, 2015.

[11] J. He, H. Yang, T. Q. Tang, and H. J. Huang, "Optimal deployment of wireless charging lanes considering their adverse effect on road capacity," *Transportation Research Part C: Emerging Technologies*, vol. 111, pp. 171–184, 2 2020.

[12] H. Liu, Y. Zou, Y. Chen, and J. Long, "Optimal locations and electricity prices for dynamic wireless charging links of electric vehicles for sustainable transportation," *Transportation Research Part E: Logistics and Transportation Review*, vol. 152, 8 2021.

[13] X. Luo and W. Fan, "Joint design of electric bus transit service and wireless charging facilities," *Transportation Research Part E: Logistics and Transportation Review*, vol. 174, 6 2023.

[14] I. Hwang, Y. J. Jang, Y. D. Ko, and M. S. Lee, "System optimization for dynamic wireless charging electric vehicles operating in a multiple-route environment," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, pp. 1709–1726, 6 2018.

[15] F. Ciari, M. Balac, and K. W. Axhausen, "Modeling carsharing with the agent-based simulation matsim: State of the art, applications, and future developments," *Transportation Research Record*, vol. 2564, no. 1, pp. 14–20, 2016.

[16] D. J. Fagnant and K. M. Kockelman, "Dynamic ride-sharing and fleet sizing for a system of shared autonomous vehicles in austin, texas," *Transportation*, vol. 45, pp. 143–158, 1 2018.

[17] M. Maciejewski and J. Bischoff, "Congestion effects of autonomous taxi fleets," *Transport*, vol. 33, pp. 971–980, 2018.

[18] G. Domingues, *Modeling, Optimization and Analysis of Electromobility Systems*. Doctoral thesis (monograph), 2018. Defence details Date: 2018-11-23 Time: 10:15 Place: Lecture hall E:1406, building E, Ole Römers väg 3, Lund University, Faculty of Engineering LTH, Lund External reviewer(s) Name: Thiringer, Torbjörn Title: Professor Affiliation: Chalmers University of Technology, Gothenburg, Sweden —.

[19] J. R. Lourenço, "Open-elevation: A free and open-source elevation api." https://open-elevation.com/, 2017. Accessed: 2025-06-05.

[20] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[21] Utah State Tax Commission, "Vehicle registrations – recent data." https://tax.utah.gov/econstats/mv/registrations, 2025. Accessed: 2025-06-10.

[22] M. Towers, A. Kwiatkowski, J. Terry, J. U. Balis, G. D. Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, H. Tan, and O. G. Younis, "Gymnasium: A standard interface for reinforcement learning environments," 2024.

[23] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[24] N. Horesh, D. A. Trinko, and J. C. Quinn, "Comparing costs and climate impacts of various electric vehicle charging systems across the united states," *Nature Communications*, vol. 15, 12 2024.

V. ADDENDUM: GPU-ACCELERATED PPO WITH GRAPHGPS

This addendum details the engineering updates that complement the joint wireless and plug-in charger placement framework. The new workflow preserves the scenario realism of MATSim [5] while reducing turnaround time and improving reproducibility through a GPU-first reinforcement learning stack built on PPO [20], PyTorch [26], PyTorch Geometric [23], and Gymnasium [22].

A. System Enhancements

We restructured *contribs/rlev* into a standalone Maven module that emits a shaded *rlev-uber.jar*. The shaded jar pins MATSim 15.0, GeoTools 29.0, and other transitive dependencies, so MATSim can be launched without a parent build. A lightweight launcher rewrites scenario configs, removes obsolete counts modules, enables EV telemetry, and enforces single-iteration runs for PPO rollouts.

MATSim now executes inside the PPO loop through a JPype bridge [27]. The bridge initializes MATSim once per experiment, injects chargers per rollout, and collects rewards through an in-memory *RewardProbe*, avoiding the socket latency and serialization overhead of the legacy reward server.

Scenario assets are sandboxed for every rollout. *MatsimXMLDataset* copies the configuration into a temporary workspace, normalizes absolute paths, tolerates *.xml.gz* inputs, and regenerates plans and vehicles when synthetic demand is requested. Outputs are written to run-scoped folders (*output_rl//*), keeping the canonical network untouched while archiving TensorBoard logs, *best_chargers.csv* files, and zipped MATSim outputs.

B. Reward Extraction and Analytics

The local controller computes rewards directly from MATSim diagnostics. Charger rewards integrate energy delivered in *0.average_charge_time_profiles.txt* and normalize by fleet battery capacity, while time penalties derive from leg-duration summaries. When telemetry is missing, the controller log provides a fallback signal. The legacy HTTP server reuses the same probe and shaded jar, ensuring parity between distributed and local experiments.

Telemetry is now exportable: TensorBoard tracks charger efficiency, cost, reward, and temporal stability, and the best-scoring configurations are persisted for downstream visualization or validation. The callback hooks into Stable-Baselines3's logging interface after each rollout, reads immutable copies of statistics from the environment *info*, and emits summaries without mutating replay buffers or optimizer state, so the PPO optimisation path remains identical to the baseline. These diagnostics eliminate the manual bookkeeping that previously accompanied PPO training.

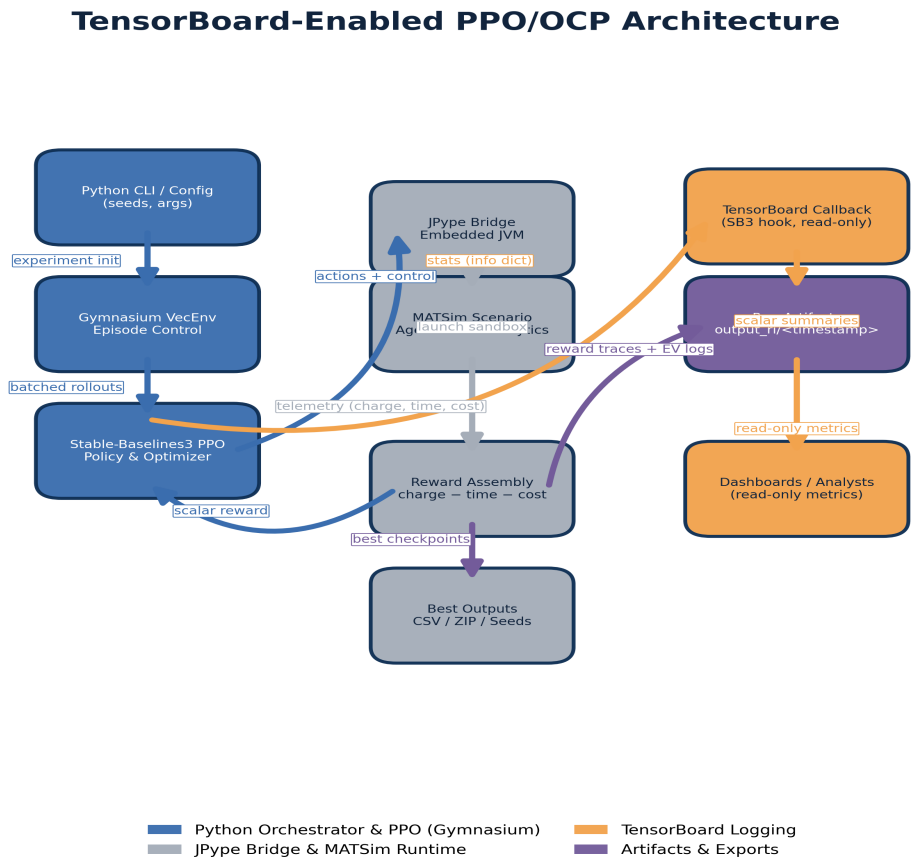

Fig. 5. Updated TensorBoard architecture. Left column (blue) shows Python + Gymnasium orchestrating PPO with MATSim. Centre (gray) captures embedded JVM execution and reward assembly. Right column (orange/violet) illustrates telemetry to TensorBoard and artifact export.

C. GraphGPS Observation Encoder

A new GraphGPS extractor [25] encodes MATSim networks into fixed-size graph representations. Each rollout selects the top-K line-graph nodes by degree, applies Graph Isomorphism Network layers with fp32-normalized residuals, and performs multi-head attention before global pooling. The encoder enables automatic mixed precision and optional CUDA graph capture, providing deterministic execution on NVIDIA hardware while clamping floating-point outliers to maintain numerical stability.

D. PPO Training Stack

The PPO runner relies on Stable-Baselines3 [20] but patches internal buffers so multi-discrete actions remain on GPU. Using *DummyVecEnv* sidesteps Windows process forking, and a comprehensive CLI exposes backend selection (*python*, *jvm*, or *server*), CUDA graph toggles, and GraphGPS hyperparameters. Each run records its arguments and aggregates per-step statistics, simplifying replication and ablation.

E. Operational Impact

The GPU-accelerated stack reduces per-iteration wall time to the MATSim runtime plus a modest constant, because graph extraction and reward computation now operate entirely in-process. Training no longer depends on 100 GB of RAM or multi-day execution windows; experiments complete in hours, and concurrent runs avoid file conflicts through sandboxing. These improvements maintain the fidelity of the original study while broadening its practicality.

REFERENCES

[25] O. Rampášek, B. G. T. Touri, B. Barekatain, et al., "Recipe for a General, Powerful, Scalable Graph Transformer," in *Advances in Neural Information Processing Systems*, 2022.

[26] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems*, 2019.

[27] JPype Contributors, "JPype User Guide," https://jpype.readthedocs.io/, accessed Sep. 2025.